附录目录:

2-1

Selected	ClusterID	Size	Silhouette	mean(Year)	Label (LSI)	Label (LLR)	Label (MI)
false	0	22	1	2022	科幻电影; 不可名状; 混沌空间;	克苏鲁 (5.66, 0.05); 克苏鲁	不可名状 (0.91); 混沌空间

					宇宙冷漠主义; 第九艺术 | 纪实性视听内容; 人工智能; web 3.0; 第九艺术; 角色设计	神话 (3.06, 0.1); 后人类 (2.01, 0.5); 网络文学 (2.01, 0.5); 恐怖文学 (2.01, 0.5)	(0.91); 小说 (0.91); 第九艺术 (0.91);
false	1	12	0.93	2021	神话叙事; 原始森林; 《盖亚》; 克苏鲁世 | 克苏鲁世; 神话叙事; 《盖亚》; 原始森林	后人类 (7.1, 0.01); 人类世 (3.46, 0.1); 神话叙事 (3.46, 0.1); 原始森林 (3.46, 0.1); 《盖亚》 (3.46, 0.1)	人类世 (0.31); 神话叙事 (0.31); 原始森林 (0.31); 《盖亚》 (0.31);
false	2	9	0.932	2022	网络文学; 神话原型; 克苏鲁神话; 神	网络文学 (7.57, 0.01); 神话原型	神话原型 (0.26); 玄幻小说 (0.26);

					话位移; 《道诡异仙》 | 《道诡异仙》; 玄幻小说; 神话原型; 网络文学; 克苏鲁神话	(3.68, 0.1); 玄幻小说 (3.68, 0.1); 神话位移 (3.68, 0.1); 疯癫 (3.68, 0.1)	神话位移 (0.26); 疯癫 (0.26);
false	4	8	0.987	2019	恐怖文学; 小众文化; 克苏鲁神话; 政治哲学; “克苏鲁神话” | “克苏鲁神话”; 洛夫克拉夫特; 政治哲学; 恐怖文学; 小众文化	恐怖文学 (7.57, 0.01); 小众文化 (3.68, 0.1); 神话学 (3.68, 0.1); “克苏鲁神话” (3.68, 0.1); 政治哲学 (3.68, 0.1)	小众文化 (0.26); 神话学 (0.26); “克苏鲁神话” (0.26); 政治哲学 (0.2
false	16	3	0.989	2018	洛夫克拉夫特; 潜意识;	毁灭 (4.95, 0.05);	毁灭 (0.12); 恐惧

					克苏鲁神话; 恐惧; 毁灭	恐惧 (4.95, 0.05); 潜意识 (4.95, 0.05); 洛夫克拉夫特 (1.46, 0.5); 克苏鲁神话 (1.46, 0.5)	(0.12); 潜意识 (0.12); 人类世 (0.08); 神话原型

2-2

Selected	**ClusterID**	**Size**	**Silhouette**	**mean(Year)**	**Label (LSI)**	**Label (LLR)**	**Label (MI)**
false	**0**	**56**	**1**	**2020**	**翻译风格; 基于语料库的; 《黄帝内经》; antconc; 《活着》 | 概念隐喻; 比较分析; 商务新闻; 创作风格;**	**语料库 (13.29, 0.001); 译者风格 (9.68, 0.005); 翻译风格 (8.49, 0.005); 概念隐喻 (5.64,**	**《双城记》 (2.29); 跨文化交际能力 (2.29); 把字句 (2.29); antconc**

					《双城记》	0.05); 语义韵 (5.49, 0.05)	
false	1	15	0.967	2018	海事英语; 语法隐喻; 功能分析; 句法位置; 语用功能 | 句法位置; 语用功能; 话语标记语; 语法隐喻; 搭配行为	语义韵 (19.63, 1.0E-4); 搭配 (14.8, 0.001); 近义词 (9.71, 0.005); 类联接 (9.71, 0.005); 海事英语 (9.71, 0.005)	表征 (0.25); 词义 (0.25); remain (0.25); anyway (0.25)
false	2	13	0.958	2018	学术写作; 学术论文; 基于语料库的; 中国研究生; 特征对比 | 人际意义; 中国形象; 政	学术写作 (10.44, 0.005); 情态动词 (10.44, 0.005); 学术论文 (10.44, 0.005); 人际意	报道动词 (0.2); 相关分析 (0.2); 特征对比 (0.2); 人称 (0.2); 功能语法

					府工作报告;语料库翻译研究;研究性文章	义(10.44, 0.005);中国形象(10.44, 0.005)	
false	3	12	0.969	2020	多维分析;译者风格;语域变异;《茶经》布里奥妮|语言特征;竞赛规则;语域风格;《孟子》英译本;译者风格	多维分析(16.87, 1.0E-4);主题词(16.87, 1.0E-4);语言特征(16.87, 1.0E-4);竞赛规则(5.55, 0.05);布里奥妮(5.55, 0.05)	竞赛规则(0.15);布里奥妮(0.15);冬奥会(0.15);《赎罪》(0.15);
false	4	12	0.92	2020	写作风格;作者识别;评价指标;语料库翻译学;思维风	文体学(25.42, 1.0E-4);思维风格(6.19, 0.05);	思维风格(0.1);语料库翻译学(0.1);评价指标

					格 | 译者行为批评; 典籍英译; 语料库翻译学; 情感反应; 情感表现	语料库翻译学 (6.19, 0.05); 评价指标 (6.19, 0.05); 叙事学 (6.19, 0.05)	(0.1); 叙事学 (0.1); 汉
false	5	9	0.975	2020	对比研究; 句法-语义界面; 事件结构; 论元实现; 互动功能 | 学术语篇; 立场标记语; 传播接受; 句法-语义界面; 互动功能	对比研究 (12.71, 0.001); 学术语篇 (12.71, 0.001); 语料库方法 (6.3, 0.05); 传播接受 (6.3, 0.05); 一带一路 (6.3, 0.05)	语料库方法 (0.09); 传播接受 (0.09); 一带一路 (0.09); 论元实现 (0.09
false	6	7	0.994	2019	叙事视角; 双语平行语料库; 第	叙事视角 (13.79, 0.001); 内视角	语料库 (0.08); 内视角 (0.07);

					一人称;翻译文体学;人称代词 | 叙事文体学;引语转换;双语平行语料库;第一人称;翻译文体学	(6.82, 0.01);限制型(6.82, 0.01);联合式(6.82, 0.01);第三人称(6.82, 0.01)	限制型(0.07);联合式(0.07);第三
false	7	7	0.98	2020	译者风格;《秦腔》译本风格;合作翻译;源语透过效应 | 源语透过效应;被动语态;译者风格;中国知网;语料库的研究方法	译者风格(32.47, 1.0E-4);刘宇昆(12.27, 0.001);语料库的研究方法(6.08, 0.05);余光中(6.08, 0.05);源语透过效应(6.08, 0.05)	语料库的研究方法(0.11);余光中(0.11);源语透过效应(0.11);中国知网(
false	8	5	0.978	2019	译	显	语

					者风格;《秦腔》译本风格；合作翻译;源语透过效应|源语透过效应；被动语态;译者风格；中国知网;语料库的研究方法	化(16.89, 1.0E-4);口译(8.28, 0.005);连接词(8.28, 0.005);《无人生还》(8.28, 0.005);语料库(0.76, 0.5)	料库(0.11);显化(0.03);口译(0.03);连接词(0.03);《无人生
false	9	4	0.986	2025	中华太极;语料库;典籍翻译；翻译共性;海外接受	海外接受(9.02, 0.005);中华太极(9.02, 0.005);典籍翻译(9.02, 0.005);翻译共性(6.27, 0.05);译者风	语料库(0.12);海外接受(0.02);中华太极(0.02);典籍翻译(0.02);

						格(0.17, 1.0)	
false	12	4	0.995	2022	泉州；城市形象；媒体话语；批评话语分析	媒体话语(9.52, 0.005); 泉州(9.52, 0.005); 批评话语分析(9.52, 0.005); 城市形象(9.52, 0.005); 语料库(1.37, 0.5)	语料库(0.13); 媒体话语(0.02); 泉州(0.02); 批评话语分析(0.02);
false	16	3	0.998	2020	语料库检索；主题；人物性格	语料库检索(10.2, 0.005); 人物性格(10.2, 0.005); 主题(10.2, 0.005); 语料库	语料库(0.14); 语料库检索(0.02); 人物性格(0.02); 主题(0.02);

						(1.02, 0.5); 译者风格 (0.1, 1.0)	
false	19	3	0.991	2022	语料库; 体育英语; 篮球英语; 术语; 词频; 词表	词表 (8.62, 0.005); 体育英语 (8.62, 0.005); 篮球英语 (8.62, 0.005); 词频 (5.87, 0.05); 术语 (5.87, 0.05)	语料库 (0.12); 词表 (0.02); 体育英语 (0.02); 篮球英语 (0.02); 《
false	21	3	0.994	2020	情感; 翻译; 语料库; 《扬子前线》; 情感词汇; 抗日战争	抗日战争 (8.62, 0.005); 《扬子前线》 (8.62, 0.005); 情感 (8.62, 0.005);	语料库 (0.12); 抗日战争 (0.02); 《扬子前线》 (0.02); 情感 (0.02);

						情感词汇 (8.62, 0.005); 翻译 (5.87, 0.05)	
false	23	3	0.998	2020	《寒夜》;《憩园》词汇特征；颜色词;句法特征	《寒夜》 (9.02, 0.005); 句法特征 (9.02, 0.005); 《憩园》 (9.02, 0.005); 词汇特征 (9.02, 0.005); 颜色词 (9.02, 0.005)	语料库 (0.13); 《寒夜》 (0.02); 句法特征 (0.02); 《憩园》 (0.02);
false	24	3	0.998	2023	基于语料库的;风格研究；文体风格;余华作品；余华小说	余华小说 (9.02, 0.005); 风格研究 (9.02, 0.005); 文体风	语料库 (0.13); 余华小说 (0.02); 风格研究 (0.02);

						格 (9.02, 0.005); 余华作品 (9.02, 0.005); 基于语料库的 (5.24, 0.05)	**文体风格 (0.02);**

2-4

Selected	ClusterID	Size	Silhouette	mean(Year)	Label (LSI)	Label (LLR)	Label (MI)
true	2	35	0.903	2019	australian english; alignment accuracy; forced alignment; bilingual corpus; engineering corpus | corpus linguistics; online lexicography; outer texts;	australian english (11.55, 0.001); lexical bundles (7.68, 0.01); corpus design (7.68, 0.01); microstructure (3.83, 0.1);	microstructure (0.54); language ideology (0.54); m

					interactional linguistics; target language	language ideology (3.83, 0.1)	
true	1	36	0.805	2019	comparable corpora; corpus-based translation studies; parallel corpora; rhetorical moves; academic texts | lexico-semantic approach; adventure tourism; corpus-based study; argument structure; frame semantics	comparable corpora (8.26, 0.005); corpus-based translation studies (8.26, 0.005); learner corpora (8.26, 0.005); learner corpus (4.71, 0.05); pragmatic marker (4.12, 0.05)	pragmatic marker (0.45); translation strategies (0
true	0	45	0.868	2020	corpus callosum; developmental neuropsych	corpus callosum (40.73, 1.0E-4); machine	persuadability (0.49); chronic effects

					ology; audiovisual translation; audience reception; interlingual subtitles | diffusion tensor imaging; white matter; traumatic brain injury; verbal memory; chronic effects	learning (7.98, 0.005); corpus callosum dysgenesis (7.98, 0.005); diffusion tensor imaging (7.98, 0.005); agenesis of the corpus callosum (7.98, 0.005)	(0.49); det
true	5	25	0.858	2020	corpus linguistics; discourse analysis; register variation; multidimensional analysis tagger; multi-dimensional analysis | critical discourse	corpus linguistics (33.25, 1.0E-4); critical discourse analysis (18.65, 1.0E-4); discourse analysis (12.39, 0.001); corpus-ass	manosphere (0.88); aboriginal people(s) (0.88); br

					analysis; corpus-assisted discourse study; online sexism; register analysis; refugees	isted discourse studies (9.27, 0.005); digital humanities (6.17, 0.05)	
true	4	31	0.875	2021	corpus literacy; teacher education; classroom teaching; teacher intention; corpus linguistics | corpus-based language pedagogy; online collaborative learning; inter-group interactions; intra-group interactions; corpus literacy	tpack (12.61, 0.001); teacher education (8.88, 0.005); corpus literacy (8.88, 0.005); corpus-based language pedagogy (7.23, 0.01); evaluation (6.27, 0.05)	evaluation (0.11); corpus-based pedagogy (0.11); l
true	7	2	0.88	2019	data-dr	data-	teacher

		0	8		iven learning; vocabulary errors; direct corpus consultation; corpus literacy; long-term corpus use | long-term corpus use; eap learner autonomy; graduate eap; it-yourself corpora; student corpus use	driven learning (17.55, 1.0E-4); teacher training (6.07, 0.05); eap learner autonomy (6.07, 0.05); student corpus use (6.07, 0.05); synthesis (6.07, 0.05)	training (0.13); eap learner autonomy (0.1
true	11	5	1	2020	deep learning; word embedding; khasi language; khasi corpus; new york times corpus | transformer model; parallel	word embedding (13.23, 0.001); deep learning (13.23, 0.001); machine translation (6.57, 0.05); khasi	machine translation (0.09); khasi corpus (0.09); d

					corpus; automatic creation; machine translation; neural machine translation	corpus (6.57, 0.05); dictionaries (6.57, 0.05)	
true	6	25	0.983	2020	gender; corpus; states; performance; feminism | statutory interpretation; courts; language; history; power	statutory interpretation (13.96, 0.001); law (13.96, 0.001); sexism (6.93, 0.01); enactment (6.93, 0.01); states (6.93, 0.01)	sexism (0.07); enactment (0.07); states (0.07); fe
true	3	34	0.851	2019	students; corpora; technology; corpus-assisted writing; data-driven learning | data-driven	corpora (8.08, 0.005); corpus linguistics (6.56, 0.05); writing (6.29,	languag e awareness (0.85); spanish l2 (0.85); micr

					learning; collocation retrieval tool; foreign language; disciplinary writing; academic purposes	0.05); corpus-based instruction (6.29, 0.05); students (5.41, 0.05)	

4-1

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from collections import Counter
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import LatentDirichletAllocation
from wordcloud import WordCloud
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
# Ensure NLTK resources are downloaded
try:
nltk.data.find('tokenizers/punkt')
nltk.data.find('corpora/stopwords')
nltk.data.find('corpora/wordnet')
nltk.data.find('taggers/averaged_perceptron_tagger')
except LookupError:
nltk.download('punkt')
```

```
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
class TextAnalyzer:
def __init__(self, stop_words=None, custom_stopwords=None, min_word_freq=2, lda_n_components=5, lda_random_state=42):
self.stop_words = set(stopwords.words('english')) if stop_words is None else set(stop_words)
self.custom_stopwords = set() if custom_stopwords is None else set(custom_stopwords)
self.min_word_freq = min_word_freq
self.lemmatizer = WordNetLemmatizer()
self.lda_n_components = lda_n_components
self.lda_random_state = lda_random_state
def preprocess_text(self, text):
# Convert to lower case
text = text.lower()
# Remove special characters and numbers
text = re.sub(r'[^a-z\s]', '', text)
# Tokenize
tokens = word_tokenize(text)
# POS Tagging and lemmatize
tokens = [self.lemmatizer.lemmatize(word, pos=self.get_wordnet_pos(word)) for word in tokens]
return tokens
def get_wordnet_pos(self, word):
"""Map POS tag to first character lemmatize() accepts"""
tag = nltk.pos_tag([word])[0][1][0].upper()
tag_dict = {"J": nltk.corpus.wordnet.ADJ,
"N": nltk.corpus.wordnet.NOUN,
"V": nltk.corpus.wordnet.VERB,
"R": nltk.corpus.wordnet.ADV}
return tag_dict.get(tag, nltk.corpus.wordnet.NOUN)
def filter_tokens(self, tokens, allowed_pos=None):
```

```
"""Filter tokens based on POS tags, stopwords and custom stopwords"""
filtered_tokens = []
if allowed_pos is not None:
pos_tags = nltk.pos_tag(tokens)
for word, pos in pos_tags:
if pos.startswith(tuple(allowed_pos)) and \
word not in self.stop_words and \
word not in self.custom_stopwords:
filtered_tokens.append(word)
else:
filtered_tokens = [word for word in tokens if word not in self.stop_words and word not in self.custom_stopwords]
return filtered_tokens
def analyze_text(self, text, allowed_pos=None):
processed_tokens = self.preprocess_text(text)
filtered_tokens = self.filter_tokens(processed_tokens, allowed_pos)
return filtered_tokens
def get_word_frequencies(self, tokens):
word_counts = Counter(tokens)
filtered_word_counts = {word: count for word, count in word_counts.items() if count >= self.min_word_freq}
return filtered_word_counts
def create_wordcloud(self, word_counts, output_path='wordcloud.png'):
wordcloud = WordCloud(width=800, height=400, background_color='white').generate_from_frequencies(word_counts)
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.savefig(output_path)
plt.show()
def apply_lda(self, texts, n_topics=5, max_features=1000):
vectorizer = TfidfVectorizer(max_features=max_features)
tfidf_matrix = vectorizer.fit_transform(texts)
```

```
lda_model = LatentDirichletAllocation(n_components=n_topics, random_state=self.lda_random_state)
lda_model.fit(tfidf_matrix)
return vectorizer, lda_model
def print_topic_words(self,vectorizer, lda_model, n_top_words=10):
feature_names = vectorizer.get_feature_names_out()
for topic_idx, topic in enumerate(lda_model.components_):
print(f"Topic {topic_idx + 1}:")
top_word_indices = topic.argsort()[:-n_top_words - 1:-1]
print(" ".join([feature_names[i] for i in top_word_indices]))
def save_analysis_results(self, word_counts, lda_results=None, output_path="analysis_results.xlsx"):
writer = pd.ExcelWriter(output_path, engine='xlsxwriter')
# Word frequency results
df_word_counts = pd.DataFrame(list(word_counts.items()), columns=['word', 'frequency'])
df_word_counts.to_excel(writer, sheet_name='word_counts', index=False)
# LDA results
if lda_results is not None:
vectorizer, lda_model = lda_results
feature_names = vectorizer.get_feature_names_out()
topics = []
for topic_idx, topic in enumerate(lda_model.components_):
top_word_indices = topic.argsort()[:-10 - 1:-1]
topics.append({f'Topic {topic_idx + 1}': " ".join([feature_names[i] for i in top_word_indices])})
df_topics = pd.DataFrame(topics)
df_topics.to_excel(writer, sheet_name='lda_topics', index=False)
writer.close()
def main():
# Sample usage:
texts = [
"""Kadath: In his house at Kadath, the ancient man Nyarlathotep,
```

```
    dreamed of the other gods. He saw the deep ones crawl from the sea and the ghouls.
    The city was old, darker than the dark, and was great
    """,
    """
    The Dream-Quest of Vellitt Boe: Vellitt Boe, the woman,
    searched for Kadath in the dreamlands, a place both marvellous and unknown.
    She saw many ghouls, travelled over the sea, and met a man who had been to the city.
    It was a dark old city, and was such that dreams of old men came from.
    """
    ]
    custom_stopwords = ['kadath', 'dream', 'man', 'city','old','saw', 'was','the']
    analyzer = TextAnalyzer(custom_stopwords=custom_stopwords,min_word_freq=1, lda_n_components=3)
    combined_tokens=[]
    filtered_texts = []
    for text in texts:
    filtered_tokens = analyzer.analyze_text(text, allowed_pos = ['NN', 'NNS','JJ', 'VBD', 'RB'])
    filtered_texts.append(" ".join(filtered_tokens))
    combined_tokens.extend(filtered_tokens)
    word_counts = analyzer.get_word_frequencies(combined_tokens)
    analyzer.create_wordcloud(word_counts, output_path="wordcloud_love.png")
    lda_results = analyzer.apply_lda(filtered_texts,n_topics=3, max_features=100)
    vectorizer, lda_model = lda_results
    analyzer.print_topic_words(vectorizer, lda_model, n_top_words=10)
    analyzer.save_analysis_results(word_counts, lda_results=lda_results, output_path="love_analysis_results.xlsx")
    print("Analysis complete. Results saved.")
    if __name__ == "__main__":
    main()
```

4-2

great: 873 seemed: 861 strange: 537 place: 518 house: 493 carter: 481 certain: 480 many: 470 black: 464 heard: 421 city: 406 earth: 389 dark: 386 light: 374 stone: 368 ancient: 365 room: 363 life: 360 door: 358 little: 351 human: 342 half: 330 began: 328 nothing: 326 horror: 319 eyes: 316 much: 302 terrible: 298 unknown: 296 sea: 286

4-3

t-SNE Component 1	t-SNE Component 2	文件名	完整书名	创作年份	主题类型	时期特征
1.290706	0.94516116	unnamable.txt	The Unnamable	1923	存在性恐怖	早期探索
2.4597273	1.0571263	mountains_of_madness.txt	At the Mountains of Madness	1931	文明探索	成熟期
2.1562092	1.5031652	dunwich.txt	The Dunwich Horror	1929	乡村恐怖	成熟期
2.6736896	0.32156473	dagon.txt	Dagon	1917	海洋神话	早期探索
2.5026937	1.2669259	festival.txt	The Festival	1925	仪式恐怖	成熟期
1.0400077	2.1977801	vault.txt	The Vault	1920	地下秘密	早期探索
2.4349017	0.92644197	pharoahs.txt	The Outsider and	1921	古代文明	早期探索

t-SNE Component 1	t-SNE Component 2	文件名	完整书名	创作年份	主题类型	时期特征
			Others			
2.26399	-0.26266	juan_romero.txt	Juan Romero	1920	文化冲突	早期探索
0.98357946	-0.270929	randolph_carter.txt	The Statement of Randolph Carter	1919	早期神话	早期探索
0.4946268	2.0979419	arthur_jermyn.txt	Arthur Jermyn	1920	种族主义	早期探索
2.1854146	0.4529689	rats_walls.txt	The Rats in the Walls	1924	社会恐怖	成熟期
3.7103999	1.5254934	sarnath.txt	The Doom that Came to Sarnath	1920	古代文明	早期探索
2.1172543	0.9705231	innsmouth.txt	The Shadow over Innsmouth	1936	种族变异	晚期
0.4708344	1.1622633	clergyman.txt	The Clergyman	1939	宗教恐怖	晚期
2.2893136	1.27224	cthulhu.txt	The Call of	1928	宇宙恐怖	成熟期

t-SNE Component 1	t-SNE Component 2	文件名	完整书名	创作年份	主题类型	时期特征
			Cthulhu			
2.1953678	1.613858	colour_out_of_space.txt	The Colour Out of Space	1927	科技恐怖	成熟期
3.195524	1.7644827	street.txt	The Street	1920	社会寓言	早期探索
1.7783989	1.8920546	gates_of_silver_key.txt	The Gate of the Silver Key	1934	维度转换	晚期
2.6294255	0.5573795	outsider.txt	The Outsider	1921	存在主义	早期探索
2.0717936	1.204848	reanimator.txt	Herbert West–Reanimator	1922	科学恐怖	早期探索
3.1662707	-0.0482377	polaris.txt	Polaris	1918	科幻萌芽	早期探索
1.9868357	1.0415154	whisperer.txt	The Whisperer in Darkness	1931	外星威胁	成熟期
3.0838792	2.496843	other_gods.txt	The Other Gods	1933	神话系统	晚期
3.724021	2.0763137	poetry_of_gods.txt	Poetry and the Gods	1920	诗歌神话	早期探索
2.33	0.86	temp	The	1920	海洋	早期

t-SNE Component 1	t-SNE Component 2	文件名	完整书名	创作年份	主题类型	时期特征
4405	96333	le.txt	Temple		恐怖	探索
1.3054094	2.3898585	descendent.txt	The Descendant	1938	家族秘密	晚期
2.3921604	1.7018845	kadath.txt	The Dream-Quest of Unknown Kadath	1927	梦境探索	成熟期
1.8809305	-0.6036959	book.txt	The Book	1938	神秘主义	晚期
3.8990889	0.05277843	what_moon_brings.txt	What the Moon Brings	1922	神秘主义	早期探索
3.2561429	0.41930184	moon_bog.txt	The Moon-Bog	1926	民间传说	成熟期
2.4605906	0.62495947	tomb.txt	The Tomb	1917	个人恐惧	早期探索
0.91693777	1.4279263	doorstep.txt	On the Doorstep	1937	社会寓言	晚期
2.5129151	0.89941764	lurking_fear.txt	The Lurking Fear	1923	集体恐惧	早期探索
2.3563504	0.83223826	shadow_out_of_time.txt	The Shadow Out of Time	1934	时间哲学	晚期

t-SNE Component 1	t-SNE Component 2	文件名	完整书名	创作年份	主题类型	时期特征
1.8690716	1.7156657	dreams_in_the_witch.txt	Dreams in the Witch House	1933	多元宇宙	晚期
3.6619236	0.9644721	nyarlathotep.txt	Nyarlathotep	1920	神话构建	早期探索
2.5679057	1.9763333	high_house_mist.txt	The High House in the Mist	1931	神秘主义	晚期
3.1256745	1.7534229	old_folk.txt	The Old Folk	1927	民间传说	成熟期
1.3637315	0.3755898	erich_zann.txt	The Music of Erich Zann	1922	音乐恐怖	早期探索
3.0903776	1.3228304	martins_beach.txt	Martin’s Beach	1923	海洋传说	早期探索
1.9184719	1.4997308	charles_dexter_ward.txt	The Case of Charles Dexter Ward	1927	科学神秘	成熟期
4.275171	1.3294238	memory.txt	Memory	1919	心理恐怖	早期探索
1.5788442	0.35339704	hypnos.txt	Hypnos	1923	意识探索	早期探索
2.29	1.07	shun	The	1924	地方	成熟

t-SNE Component 1	t-SNE Component 2	文件名	完整书名	创作年份	主题类型	时期特征
30396	48701	ned_house.txt	Shunned House		神话	期
0.93637663	0.81054676	pickman.txt	Pickman's Model	1927	艺术恐怖	成熟期
1.682763	0.10212281	picture_house.txt	The Picture in the House	1920	地方恐怖	早期探索
3.9619126	0.57096887	ex_oblivione.txt	Ex Oblivione	1921	意识边界	早期探索
2.9703176	-0.2760752	hound.txt	The Hound	1924	哥特式恐怖	早期探索
2.6218338	2.5800548	ulthar.txt	The Cats of Ulthar	1920	神话寓言	早期探索
0.69236004	0.40493074	from_beyond.txt	From Beyond	1934	认知边界	晚期
2.4671674	-0.0295342	beast.txt	The Beast in the Cave	1918	自然恐怖	早期探索
2.1574104	2.4409297	celephais.txt	Celephais	1922	梦境世界	早期探索
2.9857938	0.79826164	crawling_chaos.txt	The Crawling Chaos	1920	神话构建	早期探索
1.4290854	2.7607453	terrible_old_man.txt	The Terrible Old Man	1921	神秘主义	早期探索

t-SNE Component 1	t-SNE Component 2	文件名	完整书名	创作年份	主题类型	时期特征
3.5620008	2.7162356	tree.txt	The Tree	1920	自然神秘	早期探索
1.8290349	3.0843678	azathoth.txt	Azathoth	1922	宇宙虚无	早期探索
2.4596057	0.30195087	alchemist.txt	The Alchemist	1916	哥特式恐怖	早期探索
1.2874041	1.281376	beyond_wall_of_sleep.txt	Beyond the Wall of Sleep	1919	意识探索	早期探索
1.9945762	1.0451652	he.txt	He	1925	都市恐怖	成熟期
1.4108325	1.0087194	medusas_coil.txt	Medusa’s Coil	1939	神话恐怖	晚期
2.1884248	1.6403899	haunter.txt	The Haunter of the Dark	1935	神秘主义	晚期
1.7457044	2.0268836	silver_key.txt	The Silver Key	1929	维度探索	成熟期
2.621496	0.98188066	nameless.txt	The Nameless City	1921	探索恐怖	早期探索
1.3851801	1.6139352	cool_air.txt	Cool Air	1926	科学恐怖	成熟期
3.2639024	1.0546787	white_ship.txt	The White	1919	梦幻叙事	早期探索

t-SNE Component 1	t-SNE Component 2	文件名	完整书名	创作年份	主题类型	时期特征
			Ship			
2.611383	3.0406895	iranon.txt	The Quest of Iranon	1921	梦境探索	早期探索

4-4

```
import os
import re
import nltk
import numpy as np
import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
class LovecraftCorpusAnalyzer:
def __init__(self, corpus_path):
"""
洛夫克拉夫特语料库分析工具
Args:
corpus_path (str):  语料库路径
"""
self.corpus_path = corpus_path
self.texts, self.filenames = self._load_texts()
def _load_texts(self):
"""加载文本语料库"""
```

```
texts = []
filenames = []
for filename in os.listdir(self.corpus_path):
if filename.endswith('.txt'):
with open(os.path.join(self.corpus_path, filename), 'r', encoding='utf-8') as f:
texts.append(f.read())
filenames.append(filename)
return texts, filenames
def extract_characters(self, text):
"""
提取文本中的人物名称
Args:
text (str): 输入文本
Returns:
list: 人物名称列表
"""
使用命名实体识别
tokens = nltk.word_tokenize(text)
named_entities = nltk.ne_chunk(nltk.pos_tag(tokens))
characters = []
for subtree in named_entities:
if type(subtree) == nltk.tree.Tree:
if subtree.label() == 'PERSON':
characters.append(' '.join([leaf[0] for leaf in subtree]))
return list(set(characters))
def text_clustering(self, n_clusters=3):
"""
文本主题聚类分析
Args:
n_clusters (int): 聚类数量
Returns:
dict: 聚类结果
"""
vectorizer = TfidfVectorizer(max_features=1000)
```

```
tfidf_matrix = vectorizer.fit_transform(self.texts)
PCA 降维
pca = PCA(n_components=2)
tfidf_reduced = pca.fit_transform(tfidf_matrix.toarray())
K-means 聚类
kmeans = KMeans(n_clusters=n_clusters)
clusters = kmeans.fit_predict(tfidf_reduced)
cluster_results = {}
for i, cluster in enumerate(clusters):
if cluster not in cluster_results:
cluster_results[cluster] = []
cluster_results[cluster].append(self.filenames[i])
return cluster_results
def character_network_analysis(self):
"""
角色网络分析
Returns:
dict: 角色网络分析结果
"""
character_networks = {}
for text, filename in zip(self.texts, self.filenames):
characters = self.extract_characters(text)
G = nx.Graph()
G.add_nodes_from(characters)
简单的共现边
for i in range(len(characters)):
for j in range(i+1, len(characters)):
G.add_edge(characters[i], characters[j])
character_networks[filename] = {
'network': G,
'character_count': len(characters),
'centrality': nx.degree_centrality(G)
}
return character_networks
```

```
    def theme_analysis(self):
    """
    主题分析
    Returns:
    dict: 主题关键词
    """
    vectorizer = TfidfVectorizer(stop_words='english', max_features=50)
    tfidf_matrix = vectorizer.fit_transform(self.texts)
    feature_names = vectorizer.get_feature_names_out()
    tfidf_scores = tfidf_matrix.toarray().mean(axis=0)
    theme_keywords = sorted(
    zip(feature_names, tfidf_scores),
    key=lambda x: x[1],
    reverse=True
    )[10]
    return dict(theme_keywords)
    分析执行
    analyzer                                                    =
LovecraftCorpusAnalyzer('/Users/wewe/Desktop/lovecraftcorpus-master')
    文本聚类分析
    print("文本聚类结果：")
    print(analyzer.text_clustering())
    角色网络分析
    character_networks = analyzer.character_network_analysis()
    for filename, network_data in character_networks.items():
    print(f"\n{filename} 角色网络分析：")
    print(f"角色数量： {network_data['character_count']}")
    print("中心性最高的角色：",
    sorted(network_data['centrality'].items(),
    key=lambda x: x[1],
    reverse=True)[3])
    主题分析
    print("\n 主题关键词：")
    print(analyzer.theme_analysis())
```

4-5

词汇频率统计表

序号	词汇	频率
1	time	782
2	things	720
3	man	685
4	night	568
5	thing	535
6	men	497
7	place	474
8	house	412
9	city	386
10	world	385
100	ship	79
101	period	79
102	afternoon	78
103	blackness	78

104	objects	78
120	figure	73

4-6

代码示例：import os
import re
import sys
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
from typing import List, Dict
from collections import Counter
def mac_font_setup():
"""Mac 字体兼容方案"""
font_paths = [
'/System/Library/Fonts/PingFang.ttc',
'/System/Library/Fonts/STHeiti Light.ttc',
'/Library/Fonts/Arial Unicode.ttf'
]
for font_path in font_paths:
if os.path.exists(font_path):
matplotlib.rcParams['font.family'] = 'sans-serif'
matplotlib.rcParams['font.sans-serif'] = [font_path]
matplotlib.rcParams['axes.unicode_minus'] = False
return True
return False
设置字体
mac_font_setup()
class LovecraftAnalyzer:
def __init__(self, corpus_path: str):

```
self.corpus_path = corpus_path
self.theme_keywords = {
"cosmic_horror": [
"horror", "darkness", "alien", "unknown",
"abyss", "mysterious", "nameless"
],
"psychological_terror": [
"fear", "terror", "nightmare", "madness",
"dread", "paranoia"
]
}
def safe_load_texts(self) -> List[str]:
"""安全文本加载"""
texts = []
try:
for filename in os.listdir(self.corpus_path):
filepath = os.path.join(self.corpus_path, filename)
if filename.endswith(('.txt', '.md')):
try:
with open(filepath, 'r', encoding='utf-8') as f:
content = f.read()
if content:
texts.append(content)
except Exception as e:
print(f"文件 {filename} 读取错误： {e}")
except Exception as e:
print(f"加载文本时发生错误： {e}")
return texts
def advanced_tokenize(self, text: str) -> List[str]:
"""高级分词"""
text = re.sub(r'[\w\s]', '', text.lower())
text = re.sub(r'\d+', '', text)
stopwords = {
'the', 'a', 'an', 'and', 'or', 'but', 'in', 'on', 'at',
```

```
'to', 'for', 'of', 'with', 'by', 'from', 'up', 'about'
}
return [word for word in text.split() if word not in stopwords and len(word) > 1]
def word_frequency_analysis(self, texts: List[str]) -> Dict:
"""词频分析"""
all_words = []
for text in texts:
all_words.extend(self.advanced_tokenize(text))
word_freq = Counter(all_words)
return {
"total_unique_words": len(word_freq),
"top_words": word_freq.most_common(30),
"word_freq_distribution": word_freq
}
def theme_keyword_analysis(self, texts: List[str]) -> Dict:
"""主题关键词分析"""
theme_results = {}
for theme, keywords in self.theme_keywords.items():
occurrences = {keyword: 0 for keyword in keywords}
for text in texts:
lower_text = text.lower()
for keyword in keywords:
occurrences[keyword] += lower_text.count(keyword)
theme_results[theme] = {
"total_count": sum(occurrences.values()),
"keyword_details": dict(sorted(occurrences.items(), key=lambda x: x[1],
reverse=True))
}
return theme_results
def visualize_results(self, results: dict):
"""结果可视化"""
plt.figure(figsize=(15, 10))
词频柱状图
plt.subplot(2, 2, 1)
```

```
top_words = results['word_frequency']['top_words']
words, freqs = zip(top_words)
plt.bar(words, freqs)
plt.title('Top 30  词频', fontsize=12)
plt.xticks(rotation=90)
主题关键词柱状图
plt.subplot(2, 2, 2)
themes = list(results['theme_analysis'].keys())
theme_counts = [data['total_count'] for data in results['theme_analysis'].values()]
plt.bar(themes, theme_counts)
plt.title('主题强度分析', fontsize=12)
plt.tight_layout()
plt.show()
def comprehensive_analysis(self) -> dict:
"""综合分析"""
texts = self.safe_load_texts()
if not texts:
print("错误：未加载任何文本")
return {}
results = {
"word_frequency": self.word_frequency_analysis(texts),
"theme_analysis": self.theme_keyword_analysis(texts)
}
可视化
self.visualize_results(results)
return results
def main():
corpus_path = "/Users/wewe/Desktop/lovecraftcorpus-master"
try:
analyzer = LovecraftAnalyzer(corpus_path)
results = analyzer.comprehensive_analysis()
结果展示
for category, data in results.items():
print(f"\n{category.upper()} Analysis:")
```

```
print(data)
except Exception as e:
print(f"分析过程中发生错误：  {e}")
if __name__ == "__main__":
main()
```

4-7

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
class AdvancedDiscourseAnalyzer:
def __init__(self):
历史语境关键词 - 完整词汇
self.historical_context_keywords = {
"racial_discourse": {
"spoken_vocabulary": [
"race", "color", "ethnic", "minority", "integration",
"segregation", "prejudice", "discrimination", "equality", "heritage",
"culture", "diversity", "origins", "ancestry", "background",
"stereotype", "racism", "privilege", "identity", "inclusion",
"tribe", "nation", "homeland", "immigrant", "native",
"foreign", "mixed", "roots", "community", "lineage",
"skin", "blood", "history", "tradition", "language",
"border", "migration", "citizenship", "assimilation", "tolerance",
"bias", "whiteness", "blackness", "ethnicity", "nationality",
"persecution", "marginalize", "indigenous", "diaspora", "solidarity",
新增高敏感度词汇
"systemic_racism", "white_supremacy", "oppression", "civil_rights",
"racial_injustice", "marginalization", "black_experience", "racial_violence",
"institutional_bias", "racial_profiling", "social_inequality", "racial_tension"
],
"written_vocabulary": [
"identity", "diversity", "marginalization", "oppression", "representation",
```

"colonialism", "cultural_legacy", "social_construct", "systemic_racism", "intersectionality",

"hegemony", "subaltern", "postcolonial", "diaspora", "ethnogenesis",

"racialization", "power_dynamics", "cultural_capital", "symbolic_violence", "structural_inequality",

"racial_formation", "othering", "cultural_imperialism", "epistemic_violence", "pan-africanism",

"ethnocentrism", "cultural_hybridity", "racial_discourse", "social_stratification", "cultural_difference",

"dominant_narrative", "cultural_translation", "racial_performativity", "historical_trauma", "cultural_memory",

"social_reproduction", "symbolic_boundaries", "cultural_politics", "racial_imaginary", "epistemic_privilege",

"cultural_negotiation", "systemic_exclusion", "racial_hierarchy", "cultural_resistance", "symbolic_representation",

"social_legitimacy", "cultural_hegemony", "racial_consciousness", "institutional_racism", "cultural_autonomy",

新增深度理论词汇

"critical_race_theory", "racial_capitalism", "epistemic_violence",

"subaltern_studies", "racial_formation", "cultural_imperialism",

"racial_performativity", "settler_colonialism", "afropessimism"

]

},

"technological_anxiety": {

"spoken_vocabulary": [

"machine", "computer", "robot", "algorithm", "automation",

"AI", "digital", "internet", "network", "smart_technology",

"gadget", "software", "hardware", "chip", "data",

"code", "program", "tech", "device", "screen",

"online", "virtual", "cyber", "app", "smartphone",

"battery", "signal", "cloud", "privacy", "hack",

"virus", "security", "wireless", "interface", "platform",

"update", "server", "storage", "processor", "bandwidth",

"download", "upload", "tracking", "connection", "digital_native",

"system", "network", "firewall", "encryption", "streaming",

新增口语技术词汇

"data", "privacy", "cybersecurity", "surveillance", "innovation"

],

"written_vocabulary": [

"cybernetics", "mechanization", "technological_displacement", "algorithmic_bias", "virtual_reality",

"posthuman", "techno_dystopia", "computational_logic", "surveillance_technology", "human-machine_interface",

"technological_mediation", "digital_epistemology", "technoscientific_paradigm", "informational_capitalism", "machinic_assemblage",

"algorithmic_governance", "technological_unconscious", "digital_colonialism", "network_society", "techno_political",

"quantum_computing", "artificial_consciousness", "technological_sublime", "digital_infrastructure", "computational_thinking",

"machine_learning_ethics", "technological_determinism", "cyborg_theory", "digital_phenomenology", "technoscience",

"algorithmic_transparency", "digital_ontology", "machine_epistemology", "technological_metabolism", "informational_ecology",

"digital_materiality", "computational_culture", "techno_scientific_imaginary", "algorithmic_prediction", "digital_sovereignty",

"technological_mediation", "machine_intelligence", "digital_anthropology", "computational_social_science", "technological_unconscious",

"algorithmic_accountability", "digital_phenomenology", "technological_assemblage", "informational_power", "digital_epistemology",

新增学术技术词汇

"technological_displacement", "algorithmic_bias",

"virtual_reality", "posthuman", "computational_logic",

"surveillance_technology", "digital_epistemology",

"technological_mediation", "informational_capitalism"

]

}

}

话语指标

```
self.discourse_metrics = {
"named_entities": {
"weight": 0.3,
"marker_keywords": [
"person", "organization", "location",
"unique_identifier", "proper_noun"
]
},
"vocabulary_richness": {
"weight": 0.5,
"calculation_method": "unique_word_ratio"
},
"syntactic_depth": {
"weight": 0.2,
"marker_keywords": [
"complex_sentence", "subordinate_clause",
"embedded_structure"
]
},
"power_dynamics": {
"weight": 0.2,
"marker_keywords": [
"control", "hierarchy", "domination",
"institutional_power"
]
},
"systemic_uncertainty": {
"weight": 0.5,
"marker_keywords": [
"ambiguity", "complexity", "non_linear",
"probabilistic", "uncertain"
]
},
"othering_language": {
```

```
"weight": 0.3,
"marker_keywords": [
"exclusion", "difference", "marginalization",
"alien", "foreign"
]
}
}
def compute_metrics(self, text_corpus):
"""
计算复杂的语言和话语指标
"""
results = {}
for category, config in self.discourse_metrics.items():
metric_scores = []
for text in text_corpus:
词汇丰富度计算
if category == "vocabulary_richness":
words = text.split()
unique_words = len(set(words))
total_words = len(words)
score = unique_words / total_words if total_words > 0 else 0
关键词频率分析
else:
score = sum(
text.lower().count(keyword.lower())
for keyword in config.get('marker_keywords', [])
) / len(config.get('marker_keywords', [1]))
metric_scores.append(score)
计算统计指标
results[category] = {
"mean": np.mean(metric_scores),
"std": np.std(metric_scores)
}
return results
```

```
    def compute_historical_context_frequency(self, text_corpus):
    """
    升级版历史语境频率计算 - 动态适应
    """
    动态确定矩阵大小
    num_texts = len(text_corpus)
    frequency_matrix = np.zeros((num_texts, 2))    根据文本数量调整
    for i, text in enumerate(text_corpus):
    text_lower = text.lower()
    for j, (category, vocab_groups) in enumerate(self.historical_context_keywords.items()):
    total_keyword_matches = 0
    total_keywords = 0
    分别计算口语和书面词汇
    for vocab_type in ['spoken_vocabulary', 'written_vocabulary']:
    keywords = vocab_groups[vocab_type]
    total_keywords += len(keywords)
    高级关键词匹配
    keyword_matches = sum(
    text_lower.count(keyword.lower())
    (2 if any(sensitive_word in keyword for sensitive_word in ['systemic', 'critical', 'racial']) else 1)
    for keyword in keywords
    )
    total_keyword_matches += keyword_matches
    对数标准化，提高敏感度
    frequency = np.log1p(total_keyword_matches) / (np.log1p(total_keywords) + 1e-10)
    frequency_matrix[i, j] = frequency
    return frequency_matrix.mean(axis=0)    返回平均频率
    def advanced_visualization(self, results, text_corpus):
    """
    高级可视化 - 多维度展示
    """
```

```
        plt.figure(figsize=(20, 15))
        历史语境频率热图
        plt.subplot(2, 2, 1)
        historical_data = self.compute_historical_context_frequency(text_corpus)
        hist_df = pd.DataFrame({
        'Category': ['Racial Discourse', 'Technological Anxiety'],
        'Frequency': historical_data
        })
        sns.barplot(x='Category', y='Frequency', data=hist_df, palette='viridis')
        plt.title('Historical Context Frequency', fontsize=15)
        plt.xticks(rotation=45)
        语言复杂性图
        plt.subplot(2, 2, 2)
        linguistic_categories = ['named_entities', 'vocabulary_richness',
'syntactic_depth']
        linguistic_means = [results[cat]['mean'] for cat in linguistic_categories]
        linguistic_stds = [results[cat]['std'] for cat in linguistic_categories]
        x = np.arange(len(linguistic_categories))
        plt.bar(x - 0.2, linguistic_means, 0.4, label='Mean', color='blue')
        plt.bar(x + 0.2, linguistic_stds, 0.4, label='Std', color='orange')
        plt.title('Linguistic Complexity')
        plt.xticks(x, linguistic_categories, rotation=45)
        plt.legend()
        话语动态图
        plt.subplot(2, 2, 3)
        discourse_categories = ['power_dynamics', 'systemic_uncertainty',
'othering_language']
        discourse_means = [results[cat]['mean'] for cat in discourse_categories]
        discourse_stds = [results[cat]['std'] for cat in discourse_categories]
        x = np.arange(len(discourse_categories))
        plt.bar(x - 0.2, discourse_means, 0.4, label='Mean', color='blue')
        plt.bar(x + 0.2, discourse_stds, 0.4, label='Std', color='orange')
        plt.title('Discourse Dynamics')
        plt.xticks(x, discourse_categories, rotation=45)
```

```
plt.legend()
plt.tight_layout()
plt.show()
def analyze_corpus(self, text_corpus):
"""
主分析流程
"""
results = self.compute_metrics(text_corpus)
self.advanced_visualization(results, text_corpus)
return results
def main():
丰富的语料库
text_corpus = [
种族话语相关文本
"The complex racial dynamics of early 20th-century America reveal intricate interactions of identity, migration, and systemic racism.",
"Intersectionality and cultural legacy provide nuanced perspectives on racial discourse, challenging traditional social constructs.",
"Historical trauma and institutional racism continue to shape contemporary social stratification and power dynamics.",
"Cultural hybridity and racial performativity offer innovative lenses for understanding social complexity and marginalization.",
"The ongoing struggle for racial equality exposes deep-rooted systemic inequalities and epistemic violence.",
技术焦虑相关文本
"Technological anxiety grows with the rise of automation and AI, challenging traditional notions of human labor and intelligence.",
"The digital revolution brings unprecedented changes, creating new forms of technological mediation and computational thinking.",
"Cybernetics and machine learning challenge our understanding of human-machine interfaces and algorithmic governance.",
"Digital colonialism and informational capitalism reshape global power structures through technological infrastructure.",
```

"The emergence of quantum computing and artificial consciousness raises critical ethical questions about technological determinism."

]

analyzer = AdvancedDiscourseAnalyzer()

results = analyzer.analyze_corpus(text_corpus)

print("\n 详细分析结果： ")

for category, metrics in results.items():

print(f"\n{category} 分析： ")

for metric, value in metrics.items():

print(f" {metric}: {value:4f}")

if __name__ == "__main__":

main()

Citation:

4.2.1-3

tech_words = {

"科技词汇": [

'laboratory', 'objects', 'machine', 'ship',

'building', 'tower', 'torch', 'source'

],

"科学研究相关": [

'knowledge', 'reason', 'consciousness',

'imagination', 'facts', 'letters'

],

"技术意义词汇": [

'space', 'form', 'change', 'order',

'direction', 'use', 'key'

]

}

对应词频

{word: meaningful_words.get(word, 0) for group in tech_words.values() for word in group}

4-8

tech_words = {

"科技词汇": [
'laboratory', 'objects', 'machine', 'ship',
'building', 'tower', 'torch', 'source'
],
"科学研究相关": [
'knowledge', 'reason', 'consciousness',
'imagination', 'facts', 'letters'
],
"技术意义词汇": [
'space', 'form', 'change', 'order',
'direction', 'use', 'key'
]
}
对应词频
{word: meaningful_words.get(word, 0) for group in tech_words.values() for word in group}
4.2.1-2
{
'laboratory': 74,
'objects': 78,
'ship': 79,
'building': 83,
'tower': 81,
'torch': 74,
'source': 73,
'knowledge': 95,
'reason': 94,
'consciousness': 78,
'imagination': 74,
'letters': 87,
'space': 231,
'form': 137,
'change': 97,
'order': 89,

'direction': 93,

'use': 91,

'key': 91

}

4.2.1-3

race_words = {

"身份认同": [

'race', 'people', 'family', 'folk',

'youth', 'son', 'father', 'wife', 'uncle'

],

"社会属性": [

'city', 'town', 'village', 'region',

'land', 'home', 'streets'

],

"文化特征": [

'tales', 'legends', 'ancient',

'nature', 'books', 'memory'

]

}

对应词频

{word: meaningful_words.get(word, 0) for group in race_words.values() for word in group}

4-9

词汇使用频率表

词汇	使用频率
race	82
people	245
family	138

folk	80
youth	142
son	121
father	118
wife	84
uncle	77
city	386
town	187
village	85
region	107
land	173
home	153
streets	93
tales	124
legends	80
ancient	108

nature	141
books	107
memory	128

4-10

角色网络分析表

文件名	角色数量	中心性最高的角色
cadath.txt	141	Mount Man, Hlanith, Aphorat
book.txt	2	Tartary, Dogs
whatmoonbrings.txt	0	-
moon_bog.txt	11	Kilderry, Barry, Nemed
tomb.txt	16	Henceforward, Hiram, Betty
doorstep.txt	39	Rowley, Listen, Ipswich
lurking_fear.txt	26	Country, William Tobey, Gifford
shadowoutof_time.txt	81	Yith, Ferdinand C. Ashley, Western Australia

dreamsinthe_witch.txt	66	Professor Upham, Father Iwanicki, Heisenberg
nyarlathotep.txt	3	Trackless, Nyarlathotep, Men
highhousemist.txt	23	Cassiopeia, Mighty Ones, Shirley
old_folk.txt	31	Cassiopeia, Gothick Supremacy, Vercellius
erich_zann.txt	4	Blandot, Zann, Erich Zann
redhook.txt	49	Journals, Great Lilith, Malone
martins_beach.txt	17	Martin, James P. Orne, Faster
charlesdexterward.txt	403	Deborah B, Esek Hopkins, Ann Tillinghast Potter
memory.txt	3	Than, Rank, Memory
hypnos.txt	7	Corona Borealis, Hypnos, Hellas
shunned_house.txt	82	Archer Harris, Miss Harris, Ann White
pickman.txt	40	Boston, Listen, Copp
picture_house.txt	26	Injuns, Books, Distant
ex_oblivione.txt	1	Life

hound.txt	11	St John, Madness, Down
ulthar.txt	14	Ophir, Old Kranon, Menes
from_beyond.txt	4	Tillinghast, Space, Crawford Tillinghast
beast.txt	4	Strange, Nearer, Hope
celephais.txt	12	Nargai, Celephais, Kuranes
crawling_chaos.txt	8	De Quincey, Down, Ahead
terribleoldman.txt	12	Joe, Silva, Mr. Ricci
tree.txt	11	Musides, Tyrant, Lone
azathoth.txt	1	Noiseless
alchemist.txt	26	Barons, Le Sorcier, Count Henri
beyondwallof_sleep.txt	20	Peter Slader, Horrified, Capella
he.txt	9	Ionic, El Dorado, Greenwich
medusas_coil.txt	79	Bend Village, Atlantis, Seventh Louisiana Infantry C.S.A.
haunter.txt	48	Roderick Usher, College Street, Debris
silver_key.txt	31	Warped, Gross, Benijah

nameless.txt	22	Life, Time, Monstrous
cool_air.txt	10	Fourteenth Street, Physicians, Torres
white_ship.txt	9	Xura, High, Dorieb
iranon.txt	16	O Iranon, Kadatheron, Peasants

主题关键词:

单词	频率
old	0.20398457515009547
man	0.17183433226999356
saw	0.15884795793824097
did	0.1563761846062635
time	0.15027178796453813
like	0.1448432022330818
night	0.14456320000368716
things	0.1380646779707785
men	0.12968679409348088
came	0.12960314370409484

4-11

```
import os
import re
import nltk
import numpy as np
import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
class LovecraftCorpusAnalyzer:
def __init__(self, corpus_path):
"""
洛夫克拉夫特语料库分析工具
Args:
corpus_path (str):  语料库路径
"""
self.corpus_path = corpus_path
self.texts, self.filenames = self._load_texts()
def _load_texts(self):
"""加载文本语料库"""
texts = []
filenames = []
for filename in os.listdir(self.corpus_path):
if filename.endswith('.txt'):
with open(os.path.join(self.corpus_path, filename), 'r', encoding='utf-8') as f:
texts.append(f.read())
filenames.append(filename)
return texts, filenames
def extract_characters(self, text):
"""
```

```
提取文本中的人物名称
Args:
text (str): 输入文本
Returns:
list: 人物名称列表
"""
使用命名实体识别
tokens = nltk.word_tokenize(text)
named_entities = nltk.ne_chunk(nltk.pos_tag(tokens))
characters = []
for subtree in named_entities:
if type(subtree) == nltk.tree.Tree:
if subtree.label() == 'PERSON':
characters.append(' '.join([leaf[0] for leaf in subtree]))
return list(set(characters))
def text_clustering(self, n_clusters=3):
"""
文本主题聚类分析
Args:
n_clusters (int): 聚类数量
Returns:
dict: 聚类结果
"""
vectorizer = TfidfVectorizer(max_features=1000)
tfidf_matrix = vectorizer.fit_transform(self.texts)
PCA 降维
pca = PCA(n_components=2)
tfidf_reduced = pca.fit_transform(tfidf_matrix.toarray())
K-means 聚类
kmeans = KMeans(n_clusters=n_clusters)
clusters = kmeans.fit_predict(tfidf_reduced)
cluster_results = {}
for i, cluster in enumerate(clusters):
if cluster not in cluster_results:
```

```
cluster_results[cluster] = []
cluster_results[cluster].append(self.filenames[i])
return cluster_results
def character_network_analysis(self):
"""
角色网络分析
Returns:
dict: 角色网络分析结果
"""
character_networks = {}
for text, filename in zip(self.texts, self.filenames):
characters = self.extract_characters(text)
G = nx.Graph()
G.add_nodes_from(characters)
简单的共现边
for i in range(len(characters)):
for j in range(i+1, len(characters)):
G.add_edge(characters[i], characters[j])
character_networks[filename] = {
'network': G,
'character_count': len(characters),
'centrality': nx.degree_centrality(G)
}
return character_networks
def theme_analysis(self):
"""
主题分析
Returns:
dict: 主题关键词
"""
vectorizer = TfidfVectorizer(stop_words='english', max_features=50)
tfidf_matrix = vectorizer.fit_transform(self.texts)
feature_names = vectorizer.get_feature_names_out()
tfidf_scores = tfidf_matrix.toarray().mean(axis=0)
```

```
        theme_keywords = sorted(
        zip(feature_names, tfidf_scores),
        key=lambda x: x[1],
        reverse=True
        )[10]
        return dict(theme_keywords)
        分析执行
        analyzer                                                                    =
LovecraftCorpusAnalyzer('/Users/wewe/Desktop/lovecraftcorpus-master')
        文本聚类分析
        print("文本聚类结果：")
        print(analyzer.text_clustering())
        角色网络分析
        character_networks = analyzer.character_network_analysis()
        for filename, network_data in character_networks.items():
        print(f"\n{filename} 角色网络分析：")
        print(f"角色数量： {network_data['character_count']}")
        print("中心性最高的角色：",
        sorted(network_data['centrality'].items(),
        key=lambda x: x[1],
        reverse=True)[3])
        主题分析
        print("\n 主题关键词：")
        print(analyzer.theme_analysis())
```

5-1

woman	(0.0)
women	(0.0)
female	(0.0)
black	(-0.1667)
race	(0.0)

slave	(0.0)
tradition	(0.0)
native	(0.0)
power	(0.0)
voice	(0.0)
history	(0.0)
memory	(0.0)

5-2

Location Type	Description	Value
city	gug	5
city	ghast	4
city	ghoulish	1
town	unfamiliar	1
town	little	1
town	ulthar—a	1
town	main	1
town	high	1
town	many-turreted	1
town	small	1
village	unnamed	1
country	rough	1
country	open	1
country	gentle	1
country	green	1
place	la	2
place	silent	1
place	safe	1
place	same	1
place	own	1

place	dark	1
place	foul	1
land	inconstant	1
land	distant	1
land	own	1
mountain	great	2
valley	skai	2
valley	naraxa	1
valley	pretty	1
forest	thick	1
forest	agaric	1
desert	eastern	1
desert	green	1
sea	cerenarian	4
ocean	twilit	1

5-3

Category	Term	Value
形容词	little	11
形容词	small	9
名词	vellitt	162
名词	boe	32
名词	jurat	19
名词	cat	13
动词	had	70
动词	was	59

5-4

词性	词语	词频
形容词	great	49

形容词	old	28
形容词	other	24
形容词	black	18
形容词	last	14
形容词	such	13
形容词	many	12
形容词	marvellous	11
形容词	dark	11
形容词	unknown	10
名词	carter	273
名词	city	27
名词	ghoul	24
名词	galley	17
名词	sea	16
名词	man	14
动词	was	139
动词	had	106

动词	were	45
动词	be	43
动词	saw	34
副词	not	59
副词	so	43
副词	now	28
副词	very	22
副词	once	21

5-5

词性类别	**Kadath**	**The Dream-Quest of Vellitt Boe**
形容词 (JJ)	4395	3054
复数名词 (NNS)	3071	2143
过去式动词 (VBD)	1990	1788
副词 (RB)	1676	1245

5-6

类别	词语	值
形容词	great	49
形容词	old	28
形容词	other	24
形容词	black	18
形容词	last	14
形容词	such	13
形容词	many	12
形容词	marvellous	11
形容词	dark	11
形容词	unknown	10
名词	carter	273
名词	city	27
名词	ghoul	24
名词	galley	17
名词	sea	16
名词	man	14
动词	was	139
动词	had	106
动词	were	45
动词	be	43
动词	saw	34
副词	not	59
副词	so	43
副词	now	28
副词	very	22
副词	once	21

5-7

Filename	Word Entropy	Sentence Entropy	POS Diversity
mountains_of_madness.txt	4.34036592	10.5133294	0.22015193
unnamable.txt	4.35197881	6.5849625	0.44662713
dunwich.txt	4.38371426	9.8653075	0.30356832
dagon.txt	4.31076168	6.357552	0.46313885
festival.txt	4.31872103	6.9068906	0.40541279
vault.txt	4.33378281	7.03559153	0.43139641
pharoahs.txt	4.36390892	8.8721712	0.34295051
juan_romero.txt	4.37613765	6.96074293	0.46105106
randolph_carter.txt	4.40420534	6.357552	0.43627867
arthur_jermyn.txt	4.36511946	7.169925	0.39076087
rats_walls.txt	4.36123983	8.72036286	0.347865
sarnath.txt	4.29296576	6.53915881	0.36919592
innsmouth.txt	4.38422962	10.3827864	0.25446445
clergyman.txt	4.39599113	6.74943813	0.47148988
cthulhu.txt	4.3799511	8.85707397	0.33429566
colour_out_of_space.txt	4.30411963	9.59927382	0.28638536
street.txt	4.27754124	6.42626475	0.4011976
gates_of_silver_key.txt	4.39272186	9.27549152	0.29532328
outsider.txt	4.30181549	6.40939094	0.43549648
reanimator.txt	4.34629613	9.02982068	0.30463356
polaris.txt	4.3302322	5.72792045	0.44950495
whisperer.txt	4.37925828	10.4935266	0.24480423
other_gods.txt	4.33568165	6.40791098	0.37954665
poetry_of_gods.txt	4.31295411	5.9068906	0.4619507
temple.txt	4.34712315	7.81409216	0.37706746
descendent.txt	4.29513442	5.77160202	0.49245902
kadath.txt	4.28038131	10.3793369	0.18570689

book.txt	4.299102	5.6302184	0.4930192
what_moon_brings.txt	4.26709733	4.24792751	0.51645207
moon_bog.txt	4.27316408	6.78135971	0.3554058
tomb.txt	4.32625278	7.0768156	0.4133269
doorstep.txt	4.40311676	9.1629316	0.31562321
lurking_fear.txt	4.32435198	8.52736177	0.3584777
shadow_out_of_time.txt	4.37673351	10.0985712	0.25603438
dreams_in_the_witch.txt	4.33498026	9.14609662	0.28360454
nyarlathotep.txt	4.3176907	5.92142865	0.53097345
high_house_mist.txt	4.27694826	6.74146699	0.37617209
old_folk.txt	4.37629547	7.09726767	0.45031797
erich_zann.txt	4.31011159	6.98868469	0.37547115
redhook.txt	4.33903224	7.98299357	0.39462272
martins_beach.txt	4.32231007	6.58796697	0.48422381
charles_dexter_ward.txt	4.37392766	11.2919388	0.21070497
memory.txt	4.2804702	4.169925	0.59943182
hypnos.txt	4.30958539	6.50779464	0.42601388
shunned_house.txt	4.35047313	8.47997191	0.34417293
pickman.txt	4.39998956	7.72792045	0.36911628
picture_house.txt	4.37594321	6.91886324	0.45009416
ex_oblivione.txt	4.28014109	4.45943162	0.49643367
hound.txt	4.3393925	6.8474484	0.44832307
ulthar.txt	4.27354067	5.5849625	0.43791822
from_beyond.txt	4.34451134	7.36376124	0.42883346
beast.txt	4.28889633	6.45943162	0.43830645
celephais.txt	4.24754875	6.12928302	0.39244533
crawling_chaos.txt	4.29223638	6.87400095	0.43459351
terrible_old_man.txt	4.33835677	5.58562591	0.49720149
tree.txt	4.30468763	5.857981	0.44952894
azathoth.txt	4.2078398	3.71704271	0.60606061
alchemist.txt	4.29059097	6.84549005	0.3968988

beyond_wall_of_sleep.txt	4.3279198	7.17990909	0.41326173
he.txt	4.33429636	7.0390625	0.42853762
medusas_coil.txt	4.37714275	9.55698389	0.26910867
haunter.txt	4.36976935	9.061873	0.35953947
silver_key.txt	4.298707	7.53169895	0.38332296
nameless.txt	4.27913093	7.32192809	0.35433863
cool_air.txt	4.33301209	7.17184813	0.44437905
white_ship.txt	4.27322059	6.4429435	0.37262658
iranon.txt	4.28541163	6.40939094	0.34139985

5-8

指标 (Metric)	数值 (Value)
R 平方 (R-squared)	0.913
调整 R 平方 (Adj. R-squared)	0.895
F 统计量 (F-statistic)	65.02
F 统计量概率 (Prob (F-statistic))	4.81e-33
观测数 (No. Observations)	81
残差自由度 (Df Residuals)	68
模型自由度 (Df Model)	12
AIC	951.3
BIC	982.5

机器学习代码:

```
from transformers import BertModel, BertTokenizer
import os
import torch
import numpy as np
from sklearn.cluster import KMeans
# 指定本地保存的模型和分词器的路径
model_path = "/Users/wewe/Desktop/机器学习/base-uncased"
```

```
# 从本地加载分词器和模型
tokenizer = BertTokenizer.from_pretrained(model_path, local_files_only=True)
model = BertModel.from_pretrained(model_path, local_files_only=True)
print(f"模型和分词器已成功从本地路径 {model_path} 加载！")
# 指定书籍目录
books_directory = "/Users/wewe/Desktop/book_corpora"
# 获取目录下所有书籍文件
book_files = [os.path.join(books_directory, f) for f in os.listdir(books_directory) if f.endswith('.txt')]
# 读取每个文件的内容
book_contents = []
book_names = []
for file_path in book_files:
with open(file_path, 'r', encoding='utf-8') as file:
content = file.read()
book_contents.append(content)
book_names.append(os.path.basename(file_path))
print(f"成功读取 {len(book_contents)} 本书籍的内容。")
# 定义一个函数来生成嵌入向量
def get_embedding(text, tokenizer, model, max_length=512):
embeddings = []
for i in range(0, len(text), max_length):
segment = text[i:i + max_length]
inputs = tokenizer(segment, return_tensors="pt", max_length=max_length, truncation=True)
outputs = model(**inputs)
segment_embedding = outputs.last_hidden_state.mean(dim=1).detach().numpy()
embeddings.append(segment_embedding)
return np.mean(embeddings, axis=0)
# 生成每本书的嵌入向量
book_embeddings = []
for content in book_contents:
embedding = get_embedding(content, tokenizer, model)
book_embeddings.append(embedding)
```

```
print(f"成功生成 {len(book_embeddings)} 本书籍的嵌入向量。")
# 将嵌入向量转换为 numpy 数组
book_embeddings = np.array(book_embeddings).reshape(len(book_embeddings),
-1)
# 使用 K-means 聚类算法进行主题分类
num_clusters = 5  # 假设我们希望将书籍分为 5 个主题
kmeans = KMeans(n_clusters=num_clusters, random_state=42)
kmeans.fit(book_embeddings)
# 获取每个书籍的聚类标签
labels = kmeans.labels_
# 打印每个书籍的主题分类
for i, (book_name, label) in enumerate(zip(book_names, labels)):
print(f"书籍： {book_name}，主题： {label}")
5-9
import pandas as pd
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from collections import Counter
import re
from textblob import TextBlob
import matplotlib.pyplot as plt
from wordcloud import WordCloud
class ReviewAnalyzer:
def __init__(self, file_path):
self.df = pd.read_csv(file_path)
print(f"数据加载成功： 共 {len(self.df)} 条评论")
扩展停用词列表
self.stop_words = set(stopwords.words('english')).union({
介词
'in', 'on', 'at', 'to', 'for', 'with', 'by', 'from', 'of', 'about',
'into', 'through', 'after', 'over', 'between', 'under', 'beyond',
冠词
'a', 'an', 'the',
```

```
代词
'i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves',
'you', 'your', 'yours', 'yourself', 'yourselves',
'he', 'him', 'his', 'himself', 'she', 'her', 'hers', 'herself',
'it', 'its', 'itself', 'they', 'them', 'their', 'theirs', 'themselves',
常用动词和助动词
'be', 'is', 'am', 'are', 'was', 'were', 'been', 'being',
'have', 'has', 'had', 'having',
'do', 'does', 'did', 'doing',
'will', 'would', 'shall', 'should', 'can', 'could',
'may', 'might', 'must', 'ought', 'need',
其他虚词
'so', 'very', 'just', 'more', 'most', 'some', 'any', 'such',
'no', 'nor', 'not', 'only', 'own', 'same', 'than'
})
主题相关关键词
self.themes_keywords = {
"后殖民主义": [
"colonial", "colonialism", "postcolonial", "imperialism",
"oppression", "marginalization", "subaltern", "empire",
"resistance", "identity", "diaspora", "hybridity"
],
"女权主义": [
"feminism", "feminist", "gender", "patriarchy", "oppression",
"equality", "empowerment", "sexism", "misogyny", "women's rights"
],
"赛博女权主义": [
"cyberfeminism", "technology", "digital", "online",
"virtual", "cyborg", "technoculture", "internet",
"gender technology", "digital identity"
]
}
def clean_text(self, text):
"""清理文本数据"""
```

```
text = str(text).lower()
text = re.sub(r'[\w\s]', '', text)      移除标点符号
text = re.sub(r'\s+', ' ', text).strip()      移除多余空格
return text
def sentiment_analysis(self):
"""返回评论的情感分析结果"""
sentiments = []
sentiment_details = {
'positive': 0,
'neutral': 0,
'negative': 0
}
for review in self.df['review']:
sentiment = TextBlob(self.clean_text(review)).sentiment.polarity
sentiments.append(sentiment)
分类情感
if sentiment > 0:
sentiment_details['positive'] += 1
elif sentiment < 0:
sentiment_details['negative'] += 1
else:
sentiment_details['neutral'] += 1
return {
'average': sum(sentiments) / len(sentiments),
'details': sentiment_details
}
def analyze_word_frequency(self):
"""获取高频词汇并过滤停用词"""
words = []
for review in self.df['review']:
分词并过滤
filtered_words = [
word for word in word_tokenize(self.clean_text(review))
if word not in self.stop_words and len(word) > 1
```

```
]
words.extend(filtered_words)
return Counter(words).most_common(100)     获取前 100 个高频词
def analyze_theme_keywords(self):
"""统计与主题相关的关键词频率"""
theme_counts = {theme: [] for theme in self.themes_keywords.keys()}
for review in self.df['review']:
clean_review = self.clean_text(review)
for theme, keywords in self.themes_keywords.items():
theme_words = [
keyword for keyword in keywords
if keyword in clean_review
]
if theme_words:
theme_counts[theme].extend(theme_words)
return {theme: len(words) for theme, words in theme_counts.items()}
def generate_wordcloud(self):
"""生成词云"""
过滤停用词
text = ' '.join([
' '.join([word for word in word_tokenize(self.clean_text(review))
if word not in self.stop_words])
for review in self.df['review']
])
wordcloud = WordCloud(
width=800,
height=400,
background_color='white'
).generate(text)
plt.figure(figsize=(16,10), dpi=100)
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('评论词云')
plt.tight_layout(pad=0)
```

```
plt.savefig('wordcloud.png')
plt.close()
print("词云图已生成： wordcloud.png")
def generate_report(self):
"""生成分析报告"""
sentiment_results = self.sentiment_analysis()
freq = self.analyze_word_frequency()
theme_counts = self.analyze_theme_keywords()
report = " 后殖民主义与女权主义读者反映分析报告\n\n"
情感分析部分
report += " 情感分析\n"
report += f"- 平均情感得分： {sentiment_results['average']:2f}\n"
report += f"- 正面评论： {sentiment_results['details']['positive']} 条\n"
report += f"- 中性评论： {sentiment_results['details']['neutral']} 条\n"
report += f"- 负面评论： {sentiment_results['details']['negative']} 条\n\n"
高频词汇部分
report += " 高频词汇\n"
report += "| 词汇 | 频率 |\n"
report += "|------|------|\n"
for word, count in freq:
report += f"| {word} | {count} |\n"
主题关键词统计部分
report += "\n 主题关键词统计\n"
report += "| 主题 | 关键词计数 |\n"
report += "|------|------------|\n"
for theme, count in theme_counts.items():
report += f"| {theme} | {count} |\n"
写入报告文件
with open('postcolonial_analysis_report.md', 'w', encoding='utf-8') as f:
f.write(report)
print("报告已生成： postcolonial_analysis_report.md")
def main():
file_path = '/Users/wewe/xingoodreads_reviews.csv'
analyzer = ReviewAnalyzer(file_path)
```

```
analyzer.generate_report()
analyzer.generate_wordcloud()     生成词云
if __name__ == "__main__":
main()

import asyncio
from playwright.async_api import async_playwright
import csv
from bs4 import BeautifulSoup
import logging
import random
logging.basicConfig(level=logging.INFO,                format='%(asctime)s
- %(levelname)s: %(message)s')
async def scrape_goodreads_reviews(url, max_reviews=3000):
reviews = []
async with async_playwright() as p:
try:
browser = await p.chromium.launch(
headless=False,
args=['--disable-blink-features=AutomationControlled']
)
context = await browser.new_context(
viewport={'width': 1920, 'height': 1080},
user_agent='Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.114 Safari/537.36'
)
page = await context.new_page()
await page.set_default_timeout(120000)
logging.info(f"访问页面：  {url}")
await page.goto(url, wait_until='networkidle', timeout=120000)
处理可能的 cookie 弹窗
try:
await page.get_by_role('button', name='Close').click(timeout=5000)
except:
pass
```

```
        模拟人类滚动
        for _ in range(3):
        await page.mouse.wheel(0, random.randint(500, 1500))
        await asyncio.sleep(random.uniform(1.5, 3.5))
        page_num = 1
        while len(reviews) < max_reviews:
        logging.info(f"正在处理第 {page_num} 页")
        等待评论内容加载
        await page.wait_for_selector('section.ReviewCard__content', timeout=30000)
        content = await page.content()
        soup = BeautifulSoup(content, 'html.parser')
        精确选择评论内容
        review_elements = soup.select('section.ReviewCard__content
section.ReviewText span.Formatted')
        for review_elem in review_elements:
        if len(reviews) >= max_reviews:
        break
        review_text = review_elem.get_text(strip=True)
        if review_text and review_text not in [r['review'] for r in reviews]:
        reviews.append({'review': review_text})
        多种方式尝试点击"显示更多评论"按钮
        try:
        方法 1：使用测试 ID
        load_more_button = page.get_by_test_id('loadMore')
        if await load_more_button.count() > 0:
        await load_more_button.click()
        else:
        方法 2：使用精确的 xpath
        load_more_xpath = '//button[contains(@class, "Button--secondary")
and .span[contains(text(), "显示更多评论")]]'
        xpath_button = page.locator(load_more_xpath)
        if await xpath_button.count() > 0:
        await xpath_button.click()
        else:
```

```
logging.info("未找到'显示更多评论'按钮")
break
等待新内容加载
await page.wait_for_load_state('networkidle', timeout=30000)
await asyncio.sleep(random.uniform(2, 5))
page_num += 1
except Exception as e:
logging.warning(f"加载更多评论时出错：  {e}")
break
安全机制：如果连续多次没有新评论，则退出
if page_num > 10:     最多尝试 10 页
logging.info("已达到最大页数限制")
break
except Exception as e:
logging.error(f"抓取过程中发生致命错误：  {e}")
finally:
if reviews:
await save_reviews_to_csv(reviews)
await browser.close()
return reviews
async def save_reviews_to_csv(reviews, filename='goodreads_reviews.csv'):
if not reviews:
logging.warning("没有评论可保存")
return
try:
with open(filename, 'w', newline='', encoding='utf-8-sig') as file:
writer = csv.DictWriter(file, fieldnames=['review'])
writer.writeheader()
writer.writerows(reviews)
logging.info(f"成功保存 {len(reviews)} 条评论到 {filename}")
except Exception as e:
logging.error(f"保存评论时发生错误：  {e}")
async def main():
```

book_url =
'https://www.goodreads.com/book/show/2582189/reviews?reviewFilters=eyJhZnRlciI6Ik5qQTRMREUxTURVek5USXhNemt3TURBIn0%3D'

reviews = await scrape_goodreads_reviews(book_url, max_reviews=3000)

logging.info(f"共抓取 {len(reviews)} 条评论")

if __name__ == '__main__':

asyncio.run(main())

5-10

赛博女权主义文学分析系统 - 优化版

功能模块:

1. 增强错误处理和日志输出
2. 分块处理大文件避免内存溢出
3. 进度提示和性能优化

"""

```
import os
import re
import sys
import time
import jieba
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from tqdm import tqdm
from wordcloud import WordCloud
from snownlp import SnowNLP
from sklearn.feature_extraction.text import TfidfVectorizer
# ================= 配置模块 =================
plt.rcParams['font.sans-serif'] = ['Arial Unicode MS']  # Mac 中文显示
plt.rcParams['axes.unicode_minus'] = False
# ================= 工具函数 =================
class FileProcessor:
@staticmethod
def safe_read(file_path):
"""安全读取文件，支持多种编码"""
```

```
encodings = ['utf-8', 'gb18030', 'gbk']
for enc in encodings:
try:
with open(file_path, 'r', encoding=enc) as f:
return f.read()
except UnicodeDecodeError:
continue
except FileNotFoundError:
raise
raise ValueError(f"无法解码文件： {file_path}")
@staticmethod
def clean_text(text):
"""优化的文本清洗"""
text = re.sub(r'\n{2,}', '\n', text)  # 保留单个换行
text = re.sub(r'[\u4e00-\u9fa5.,!?、;:“”‘’()《》 【】\n]+', '', text)
return text.strip()
@staticmethod
def chunk_processing(text, chunk_size=100000):
"""分块处理大文本"""
return [text[i:i+chunk_size] for i in range(0, len(text), chunk_size)]
class AnalysisEngine:
def __init__(self, config):
self.config = config
self.stopwords = self._load_lexicon('stopwords')
self.lexicons = {
'tech_ethics': self._load_lexicon('tech_ethics'),
'cthulhu': self._load_lexicon('cthulhu_lexicon'),
'mother_daughter': self._load_lexicon('mother_daughter')
}
self.data = pd.DataFrame()
def _load_lexicon(self, key):
"""加载词典文件"""
path = self.config[key]
if not Path(path).exists():
```

```
raise FileNotFoundError(f"词典文件不存在： {path}")
return set(Path(path).read_text(encoding='utf-8').splitlines())
def process_novels(self, novel_paths):
"""处理小说文件主要流程"""
records = []
print("\n 开始处理小说文件……")
for path in tqdm(novel_paths, desc='处理进度', file=sys.stdout):
try:
start_time = time.time()
filename = Path(path).name
print(f"\n▶ 正在处理： {filename}")
# 读取和清洗文本
raw_text = FileProcessor.safe_read(path)
cleaned_text = FileProcessor.clean_text(raw_text)
# 分块处理
chunks = FileProcessor.chunk_processing(cleaned_text)
chunk_results = []
for chunk in chunks:
# 分词和词性标注
words = [word for word in jieba.cut(chunk)
if word not in self.stopwords and len(word) > 1]
# 特征提取
chunk_results.append({
'feminism_score': sum(1 for w in words if w in self.lexicons['mother_daughter']),
'cthulhu_score': sum(1 for w in words if w in self.lexicons['cthulhu']),
'tech_score': sum(1 for w in words if w in self.lexicons['tech_ethics']),
'word_count': len(words)
})
# 聚合分块结果
avg_scores = pd.DataFrame(chunk_results).mean().to_dict()
# 元数据提取
meta = self._extract_metadata(filename)
records.append({
'title': meta['title'],
```

```
'year': meta['year'],
**avg_scores,
'processing_time': time.time() - start_time
})
except Exception as e:
print(f"处理文件 {filename} 时出错: {str(e)}")
continue
self.data = pd.DataFrame(records)
return self
def _extract_metadata(self, filename):
"""提取年份和标题"""
match = re.search(r'(.+?)-(d{4})\.txt', filename)
if match:
return {
'title': match.group(1).replace(' ', ''),
'year': int(match.group(2))
}
return {'title': filename, 'year': 0}
def generate_report(self):
"""生成分析报告"""
# 趋势分析
self._plot_trend()
# 对比词云
self._generate_wordclouds()
# 保存 Markdown 报告
self._save_markdown()
print("\n 分析完成！结果已保存至以下文件：")
print("- novel_analysis.md")
print("- trend_analysis.png")
print("- comparison_wordcloud.png")
def _plot_trend(self):
"""生成趋势分析图"""
plt.figure(figsize=(12, 6))
for col in ['feminism_score', 'cthulhu_score', 'tech_score']:
```

```
yearly = self.data.groupby('year')[col].mean()
plt.plot(yearly.index, yearly.values, marker='o', label=col)
plt.title('元素演变趋势  (2014-2024)')
plt.xlabel('年份')
plt.ylabel('标准化得分')
plt.legend()
plt.grid(True)
plt.savefig('trend_analysis.png', dpi=300)
plt.close()
def _generate_wordclouds(self):
"""生成对比词云"""
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))
#  女权相关词云
feminism_text = ' '.join(self.data[self.data['feminism_score'] > 3]['title'])
wc1 = WordCloud(font_path='Arial Unicode.ttf', width=800, height=600,
colormap='Reds').generate(feminism_text)
ax1.imshow(wc1)
ax1.set_title('女权元素相关作品', fontsize=14)
ax1.axis('off')
#  克苏鲁相关词云
cthulhu_text = ' '.join(self.data[self.data['cthulhu_score'] > 3]['title'])
wc2 = WordCloud(font_path='Arial Unicode.ttf', width=800, height=600,
colormap='Blues').generate(cthulhu_text)
ax2.imshow(wc2)
ax2.set_title('克苏鲁元素相关作品', fontsize=14)
ax2.axis('off')
plt.tight_layout()
plt.savefig('comparison_wordcloud.png', dpi=300)
plt.close()
def _save_markdown(self):
"""保存 Markdown 报告"""
content = [
"#  赛博女权主义文学分析报告\n",
"##  总体趋势\n",
```

```
f"![趋势图](trend_analysis.png)\n",
"## 作品对比\n",
f"![词云对比](comparison_wordcloud.png)\n",
"## 详细数据\n"
]
# 添加表格
table = self.data[['title', 'year', 'feminism_score', 'cthulhu_score', 'tech_score']]
content.append(table.to_markdown(index=False))
# 添加《废土世界》专项分析
content.extend([
"\n## 《我在废土世界扫垃圾》专项分析",
"### 核心发现",
"- 非传统母女关系：白澄称祝宁为'母亲'，体现科技对亲情的重构",
"- 记忆操控主题：祝宁对祝遥记忆模型的干预反映权力动态",
"- 废土环境下的女性互助：危机中的合作突破传统性别角色\n",
"### 研究计划",
"1. 文本细读：分析关键情节中的对话描写",
"2. 比较研究：对比《异种之母》中的母性表达",
"3. 理论应用：运用赛博女性主义分析科技与性别的关系"
])
with open('novel_analysis.md', 'w', encoding='utf-8') as f:
f.write('\n'.join(content))
# ================ 主程序 ================
if __name__ == "__main__":
# 配置信息
CONFIG = {
'stopwords': '/Users/wewe/Desktop/关键词/chinese_stopwords.txt',
'tech_ethics': '/Users/wewe/Desktop/关键词/tech_ethics_lex.txt',
'cthulhu_lexicon': '/Users/wewe/Desktop/关键词/cthulhu_lexicon.txt',
'mother_daughter': '/Users/wewe/Desktop/关键词/mother_daughter_lex.txt'
}
NOVEL_PATHS = [
'/Users/wewe/Desktop/中国女频克苏鲁/怨气撞铃-2014.txt',
'/Users/wewe/Desktop/中国女频克苏鲁/非人类饲养员-2024.txt',
```

'/Users/wewe/Desktop/中国女频克苏鲁/赛博游戏-2024.txt',
'/Users/wewe/Desktop/中国女频克苏鲁/诡秘之主-2018.txt',
'/Users/wewe/Desktop/中国女频克苏鲁/我在废土世界扫垃圾-2024.txt',
'/Users/wewe/Desktop/中国女频克苏鲁/异种之母-2017.txt',
'/Users/wewe/Desktop/中国女频克苏鲁/不可名状的聊天群-2024.txt'
]
try:
# 初始化分析引擎
engine = AnalysisEngine(CONFIG)
# 处理小说文件
engine.process_novels(NOVEL_PATHS)
# 生成报告
engine.generate_report()
except Exception as e:
print(f" 系统错误：{str(e)}")
print("可能原因及解决方案：")
print("1. 文件路径错误 → 检查 CONFIG 和 NOVEL_PATHS 中的路径")
print("2. 内存不足 → 尝试分块处理或使用更小文本")
print("3. 依赖缺失 → 运行 pip install -r requirements.txt")
custom_keywords.txt
技术伦理主题词典
仿生人
觉醒
系统
叛逃者
代码
共情
机械
飞升
硅基
恋爱
云端
妈妈
母亲

控制
生育
痛苦
女儿
怀孕
自由
保护
母性
和解
妥协
生育
数据
遗产
机器人三定律
漏洞
碳基生命
优越论
电子宠物
记忆篡改
AI 抑郁症
虚拟犯罪
意识备份
感官租赁协议
人格数据
思维专利
AI
神学
算法
乌托邦
虚拟
永生
电子
戒断
神经

超频
赛博格化
意识殖民
仿生
AI 恋人
记忆芯片
情绪
代码
脑机接口
元宇宙
分身
基因编辑
仿生人
婴儿
意识上传
量子
永生
克隆
替身
数据人格
算法
囚徒
情感计算偏差
科技
人工智能
机器人
虚拟现实
互联网
数字化
人性
情感
道德
灵魂
意识

自由意志
技术伦理
伦理困境
人机交互
科技异化
虚拟与现实
技术依赖
脑机械触手
机械战士
共生
黑客
信号发送器
意识体云端
绝对服从
记忆模型
子民
精神值下降
蚕食
怪物
探测
机械
异种之母
虫卵
机械触手
机械战士
意识体云端
母爱
控制
反抗
cthulhu_adjectives.txt
不可名状的
扭曲的
非欧几里得的
令人窒息的

亵渎的
混沌的
远古的
禁忌的
畸形的
腐败的
黏腻的
蠕动的
低语般的
脉动的
非自然的
多维的
深渊般的
溃烂的
复眼状的
胶质的
非人形的
使人癫狂的
空间折叠的
时间错位的
血肉模糊的
角质化的
磷光闪烁的
膜翅振动的
腔肠结构的
节肢状的
超维度的
反逻辑的
自我复制的
星间低语的
熵增的
反因果的
伪足蠕行的
孢子散播的

神经寄生的
眼球丛生的
吸盘密布的
骨骼畸变的
液态流动的
晶体增殖的
声波共振的
记忆侵蚀的
梦境渗透的
现实撕裂的
维度重叠的
感官过载的
认知污染的
重力紊乱的
血肉共生的
伪神圣的
触须丛生的
黏液分泌的
复口器的
伪足蠕动的
相位偏移的
chinese_stopwords
中文停用词表 v2.1
适用领域：网络文学分析/自然语言处理
最后更新：2024-03-20
的 地 得 了 着 过 啊 吧 呢 吗 啦 呀 哇 哦 哎 嗯 呐 呵 哟 喂 罢 咧 呗 喽 么 兮 哉 矣 焉 耳 尔 乎 之 者 也 亦 已 以 于 而 且 但 或 乃 即 则 皆 尽 尤 犹 又 再 更 最 太 极 甚 愈 越 都 共 并 兼 及 与 跟 同 和 及 以及 或者 还是 不是 就是 便是 正是 正是 不是 就是 便是 正是 不是 就是 便是 正是
我 你 他 她 它 我们 你们 他们 她们 它们 咱 们 自己 本人 别人 人家 大家 谁 什么 哪 哪里 哪儿 如何 怎样 多少 几 这么 那么 这样 那样 其他 其余 该 本 此 某 每 各 有的 有些 所有 任何 一切 某个 某些 哪个 哪些 哪里 如何 为什么 怎么样

一 二 三 四 五 六 七 八 九 十 百 千 万 亿 零 半 两 些 个 位 件 条 只 支 匹 张 座 回 次 遍 趟 番 顿 阵 场 把 口 头 首 篇 本 页 章 节 辆 台 架 艘 列 队 群 双 对 副 套 批 组 类 种 系列 多数 少数 大量 少量 部分 整个 全部 所有 祝宁

年 月 日 时 分 秒 周 星期 今天 明天 昨天 现在 过去 未来 之前 之后 之上 之下 之前 之后 左边 右边 前面 后面 里面 外面 旁边 中间 东部 西部 南部 北部 中央 附近 周围 地区 地方 位置 方向 角度 位置 地点 区域 空间 时间 时候 时刻 时期 阶段 期间 当前 最近 最初 最后 开始 结束

， 。 、 ！ ？ ： ； “ ” ‘ ’ （ ） 《 》 〈 〉 【 】 『 』 「 」 ﹁ ﹂ 〔 〕 … — ～ · · · ￥ ￡ § ※ ☆ ★ ○ ● ◎ ◇ ◆ □ ■ △ ▲ ▽ ▼ → ← ↑ ↓ ↔ ↖ ↗ ↘ ↙ ㊣ ﹌ ﹋ ﹍ ﹎ ﹏ ＿ ˍ ▁ ▂ ▃ ▄ ▅ ▆ ▇ █ ▏ ▎ ▍ ▌ ▋ ▊ █ ◢ ◣ ◤ ◥

【置顶】 【公告】 【推荐】 点击 查看 下载 回复 楼主 沙发 板凳 地板 第N楼 第X层 用户名 密码 验证码 登录 注册 忘记密码 个人中心 我的主页 消息 通知 设置 退出 收藏 分享 点赞 转发 评论 举报 投诉 客服 帮助 关于我们 联系方式 友情链接 合作伙伴 网站地图 备案号 ICP 版权所有 © ® ™ 微信公众号 微信小程序 二维码 扫一扫 关注 订阅 会员 VIP 免费 付费 价格 优惠 促销 活动 广告 推广

元 角 分 厘 米 分米 厘米 毫米 微米 纳米 千米 公里 里 尺 寸 丈 吨 千克 克 毫克 升 毫升 磅 盎司 秒 分钟 小时 天 周 月 年 世纪 平方米 立方米 立方厘米 升 毫升 转/分 赫兹 分贝 伏特 安培 瓦特 焦耳 卡路里 摄氏度 华氏度 像素 分辨率 DPI

a an the and or but if then else when where how why which what who whom whose this that these those am is are was were be been being have has had having do does did doing will would shall should can could may might must

^[0-9]+$ # 纯数字

^[a-zA-Z]+$ # 纯英文字母

^[\u4e00-\u9fa5]+$ # 无中文字符

mother_daughter_lex

母女关系主题词典

妈妈

母亲

控制

生育

痛苦
女儿
怀孕
自由
保护
母性
和解
妥协
cthulhu_lexicon
克苏鲁主题词典
主宰者
深潜者
达贡
海德拉
星之眷族
利维坦
潮汐祭司
溺亡者之城
海底
神殿
深渊
回响
咸腥
海雾
珊瑚
尸骸
浮游生物
触手
溺亡者
低语
深渊
裂沟
海底火山口
黏液

盐渍
鱼腥
诅咒
珊瑚
深海
耳鸣
溺毙
潮汐锁定
海沟回音
发光浮游疹
盐晶
鳃裂
变异
深海失语症
清洁共生
互利共生
寄生关系
伴游关系
庇护共生
营养交换
信息传递
群体智慧
集体
迁徙
声波
交流
发光
诱导
伪装
共生
毒性免疫
记忆传承
喂食
触摸治疗

协同捕猎
舞蹈交流
育儿协助
危机预警
导航
指引
记忆传承
伤口清洁
寄生虫清除
领地共享
跨物种游戏
声呐教学
鲨鱼
水母
海豚
章鱼
珊瑚
海龟
鲸鱼
海星
海马
海葵
寄居蟹
小丑鱼
蝠鲼
海牛
海豹
海狮
海蛇
海胆
藤壶
牡蛎
水母
章鱼

SAN 值
认知
崩溃
克苏鲁之梦
阿卡姆疯人院
星空恐惧症
深海恐惧症
触手 ptsd
不可名状
理智检定
旧印护身符
记忆消除术
认知滤网
精神污染
抗性
模因
病毒
人鱼
鱼
菌类
星之彩
黄印污染
血肉畸变
基因污染
认知模因
腐化孢子
精神菌毯
眼球
增生
鳞片皮疹
鳃裂综合征
角质化皮肤
不可名状痘印
黏液

分泌
过剩
复眼
角质指甲
异变
经期
畸胎瘤
骨骼软化
声带异化
记忆
蠕虫
视网膜投影
脑沟
迥异生
共生
真菌
血液荧光化
肢体冗余
内脏位移
代谢
加速
异种之母
虫卵
卵鞘
tech_ethics_lex
技术伦理主题词典
仿生人
觉醒
系统
叛逃者
代码
共情
机械
飞升

硅基
恋爱
云端
生育
数据
遗产
机器人三定律
漏洞
碳基生命
优越论
电子宠物
记忆篡改
AI 抑郁症
虚拟犯罪
意识备份
感官租赁协议
人格数据
思维专利
AI
神学
算法
乌托邦
虚拟
永生
电子
戒断
神经
超频
赛博格化
意识殖民
仿生
AI 恋人
记忆芯片
情绪

代码
脑机接口
元宇宙
分身
基因编辑
仿生人
婴儿
意识上传
量子
永生
克隆
替身
数据人格
算法
囚徒
情感计算偏差
科技
人工智能
机器人
虚拟现实
互联网
数字化
人性
情感
道德
灵魂
意识
自由意志
技术伦理
伦理困境
人机交互
科技异化
虚拟与现实
技术依赖

脑机械触手
机械战士
共生
黑客
信号发送器
意识体云端
绝对服从
记忆模型
子民
精神值下降
蚕食
怪物
探测
机械
5-11

```
import json
import jieba
import jieba.posseg as pseg
import networkx as nx
import pandas as pd
from snownlp import SnowNLP
from gensim import corpora, models
from collections import Counter

def generate_custom_dict():
    """生成自定义词典"""
    comprehensive_terms = [
        # 克苏鲁相关学术与口语词汇
        '克苏鲁', '旧日支配者', '诡秘', '恐怖', '神秘',
        '克苏鲁神话', '外神', '深潜者', '不可名状',

        # 网络评论用语
        '牛', '蒂', '厉害', 'AI', 'intj', 'cp',
        '强', 'nb', '大女主', '同理心',
```

```
            # 文学理论术语
            '赛博女性主义', '后殖民主义', '男性',
            '权力结构', '身份认同', '女性', 'ai',

            # 读者评价词
            '好看', '有趣', '震撼', '燃', '牛批',
            '绝了', '牛呀', '猛', '太强',

            # 文学流派
            '克苏鲁', '恐怖', '赛博朋克', '蒸汽朋克',
            '悬疑', '推理', '克系', '科幻', '魔幻',

            # 专业分析术语
            '语义网络', '话语分析', '批评性话语',
            '主体性', '文化间性', '权力转换',

            # 情感与态度表达
            '酷', '炸裂', '牛批', '秀', '杠杠的',

            # 具体作品
            '诡秘之主', '赛博游戏', '黑暗幻想',
            '扫垃圾', '克苏鲁神话'
        ]
        with open('custom_dict.txt', 'w', encoding='utf-8') as f:
            for term in comprehensive_terms:
                f.write(f"{term} n 5\n")

    class ChineseTextAnalyzer:
        def __init__(self):
            generate_custom_dict()
            self.gender_keywords = ['女性', '性别', '身体', '权力', '技术', '主体']
            self.postcolonial_keywords = ['殖民', '边缘', '中心', '文化', '权力', '话
语']
```

```
        jieba.load_userdict('custom_dict.txt')

    def preprocess_text(self, texts):
        processed_texts = []
        for text in texts:
            words = pseg.cut(text)
            processed_texts.append([(word, flag) for word, flag in words])
        return processed_texts

    def semantic_network_analysis(self, processed_texts):
        G = nx.Graph()
        for text in processed_texts:
            for i in range(len(text)-1):
                G.add_edge(text[i][0], text[i+1][0])
        return G

    def gender_discourse_analysis(self, texts):
        gender_texts = [text for text in texts if any(keyword in text for keyword in self.gender_keywords)]
        keyword_freq = {keyword: sum(text.count(keyword) for text in gender_texts) for keyword in self.gender_keywords}
        return {'gender_texts': gender_texts, 'keyword_frequency': keyword_freq}

    def postcolonial_discourse_analysis(self, texts):
        marginal_texts = [text for text in texts if any(keyword in text for keyword in self.postcolonial_keywords)]
        return {'marginal_narratives': marginal_texts}

    def comprehensive_analysis(self, filepath):
        with open(filepath, 'r', encoding='utf-8') as f:
            texts = json.load(f)
        processed_texts = self.preprocess_text(texts)
        semantic_network = self.semantic_network_analysis(processed_texts)
```

```
        gender_discourse = self.gender_discourse_analysis(texts)
        postcolonial_discourse = self.postcolonial_discourse_analysis(texts)
        sentiment = self.sentiment_analysis(texts)
        topic = self.topic_modeling(texts)
        frequent = self.frequent_itemset_mining(texts)

        return {
            'semantic_network': semantic_network,
            'gender_discourse': gender_discourse,
            'postcolonial_discourse': postcolonial_discourse,
            'sentiment': sentiment,
            'topic': topic,
            'frequent': frequent
        }

    def sentiment_analysis(self, texts):
        sentiments = [SnowNLP(text).sentiments for text in texts]
        positive_count = sum(1 for s in sentiments if s > 0.5)
        negative_count = sum(1 for s in sentiments if s < 0.5)
        neutral_count = len(sentiments) - positive_count - negative_count
        return {'sentiments': sentiments, 'positive': positive_count, 'negative':
negative_count, 'neutral': neutral_count}

    def topic_modeling(self, texts):
        words = [list(jieba.cut(text)) for text in texts]
        dictionary = corpora.Dictionary(words)
        corpus = [dictionary.doc2bow(text) for text in words]
        lda = models.LdaModel(corpus, num_topics=3, id2word=dictionary,
passes=15)
        topics = lda.print_topics(num_words=10)
        return {'topics': topics}

    def frequent_itemset_mining(self, texts):
        # 定义更全面的停用词集合
```

```
        stop_words = {
            '的', ',  ', '。', '了', ' ', '是', '我', '很', '看', ' ! ',
            '、', ':  ', '"', '"', '?  ', '…', '但', '和', '都', '有',
            '这', '那', '就', '又', '会', '能', '在', '于', '对', '还',
            '吗', '呢', '啊', '哦', '哈', '呀', '吧'
        }

        # 存储有效词语
        all_meaningful_words = []

        for text in texts:
            # 精确分词，过滤停用词和单字
            words = [
                word for word in jieba.cut(text, cut_all=False)
                if word not in stop_words and len(word) > 1
            ]
            all_meaningful_words.extend(words)

        # 统计词频
        word_counts = Counter(all_meaningful_words)

        # 获取 top 20 个词
        most_common = word_counts.most_common(20)

        return {'most_common': most_common}


    def main():
        analyzer = ChineseTextAnalyzer()
        cyberpunk_results =
analyzer.comprehensive_analysis('/Users/wewe/Desktop/赛博游戏书评.json')
        mysterious_results =
analyzer.comprehensive_analysis('/Users/wewe/Desktop/诡秘之主书评.json')
```

```
        print("赛博游戏书评分析结果:")

print(pd.DataFrame(cyberpunk_results['gender_discourse']['keyword_frequency'].items(), columns=['关键词', '频率']))
        print("\n 情感分析:", cyberpunk_results['sentiment'])
        print("\n 主题建模:", cyberpunk_results['topic'])
        print("\n 频繁项集:", cyberpunk_results['frequent'])

        print("\n\n 诡秘之主书评分析结果:")

print(pd.DataFrame(mysterious_results['postcolonial_discourse']['marginal_narratives'], columns=['边缘叙事']))
        print("\n 情感分析:", mysterious_results['sentiment'])
        print("\n 主题建模:", mysterious_results['topic'])
        print("\n 频繁项集:", mysterious_results['frequent'])

    if __name__ == "__main__":
        main()

    5-12
    import json
    import jieba
    from collections import Counter
    from snownlp import SnowNLP
    class AIDiscourseAnalyzer:
    def __init__(self, filepath):
    with open(filepath, 'r', encoding='utf-8') as f:
    self.texts = json.load(f)
    AI 相关关键词
    self.ai_keywords = [
    'AI', 'ai', '人工智能', '智能',
    '机器', '计算', '算法',
    '技术', '程序', '数据'
    ]
```

```
def ai_context_analysis(self):
"""AI 语境分析"""
ai_contexts = []
for text in self.texts:
for keyword in self.ai_keywords:
if keyword in text:
ai_contexts.append(text)
return {
'total_ai_mentions': len(ai_contexts),
'example_contexts': ai_contexts[5]
}
def ai_sentiment_analysis(self):
"""AI 相关文本情感分析"""
ai_texts = []
for text in self.texts:
for keyword in self.ai_keywords:
if keyword in text:
ai_texts.append(text)
if not ai_texts:
return {
'sentiment_mean': None,
'positive_ratio': None,
'negative_ratio': None
}
sentiments = [SnowNLP(text).sentiments for text in ai_texts]
return {
'sentiment_mean': sum(sentiments) / len(sentiments),
'positive_ratio': sum(1 for s in sentiments if s > 0.5) / len(sentiments),
'negative_ratio': sum(1 for s in sentiments if s < 0.5) / len(sentiments)
}
def ai_keyword_correlation(self):
"""AI 相关关键词共现分析"""
ai_related_words = []
for text in self.texts:
```

```
分词
words = list(jieba.cut(text))
检查是否包含 AI 关键词
if any(keyword in text for keyword in self.ai_keywords):
过滤停用词
filtered_words = [
word for word in words
if len(word) > 1 and word not in self.ai_keywords]
ai_related_words.extend(filtered_words)
统计共现词
word_freq = Counter(ai_related_words)
return word_freq.most_common(10)
def comprehensive_ai_analysis(self):
"""综合 AI 话语分析"""
return {
'ai_context': self.ai_context_analysis(),
'ai_sentiment': self.ai_sentiment_analysis(),
'ai_keyword_correlation': self.ai_keyword_correlation()
}
def main():
分析两本书的 AI 话语
cyberpunk_ai_analyzer = AIDiscourseAnalyzer('/Users/wewe/Desktop/赛博游戏书评。json')
mysterious_ai_analyzer = AIDiscourseAnalyzer('/Users/wewe/Desktop/诡秘之主书评。json')
获取分析结果
cyberpunk_ai_results = cyberpunk_ai_analyzer.comprehensive_ai_analysis()
mysterious_ai_results = mysterious_ai_analyzer.comprehensive_ai_analysis()
输出分析结果
print("赛博游戏-AI 话语分析：")
print("AI 提及总数：", cyberpunk_ai_results['ai_context']['total_ai_mentions'])
print("AI 情感分析：", cyberpunk_ai_results['ai_sentiment'])
print("AI 相关关键词：", cyberpunk_ai_results['ai_keyword_correlation'])
print("\n 诡秘之主-AI 话语分析：")
```

```
print("AI 提及总数：", mysterious_ai_results['ai_context']['total_ai_mentions'])
print("AI 情感分析：", mysterious_ai_results['ai_sentiment'])
print("AI 相关关键词：", mysterious_ai_results['ai_keyword_correlation'])
if __name__ == "__main__":
main()
```